CSCI 5300 Project Planning Report

Project Title: Melody Matrix

Team Members:

Vignesh Venkatesan

Nanditha Bhushana Nagabhushana

Mattia Fiore

Mansi Gidugu

Sai Sashank Dharmireddi

1. Description:

Melody Matrix is an online platform that transforms MIDI music into a live digital jam session. As opposed to listening to a pre-recorded song, the system analyzes how music sequences by learning what notes are likely to come after one another. It relies on probability to learn the patterns in the melody and rhythm, and then applies this to form new music that will seem exactly like the old style yet never the same at all. Every time you press the play button, the system comes up with a new performance.

It is written in Python and uses MIDI files to decode musical information in terms of pitch, duration, and velocity. It transforms these facts into probability maps displaying the connection between notes. These maps are stored in MongoDB in a flexible manner in such a way that every trained model is easily stored and reused. This is because, once a user initiates a session, the trained model is transferred to the browser. React and Tone.js are used to compose the frontend which implements those probabilities in real sound on-the-fly. The server provides the storage of learning and data and the browser is the instrument.

The peculiarity of Melody Matrix is that it does not look like a single music producer, but rather a small band. It generates melody, bass, and drum simultaneously. Both parts make their own judgments according to acquired probabilities, yet all of them remain in time with the chosen tempo. The Markov order can be changed to allow users to vary the speed, select a mood e.g. calm, happy or dark, and the complexity of the music. They are also capable of training their own models via uploading their own MIDI files, saving their own works, and downloading the created music.

Melody Matrix aims at turning the music generation into an interactive, creative process and demonstrating good software design concepts. It integrates real-time audio playback, artificial intelligence, database management, and secure user accounts into a single clear system. The platform remains scalable and organized since it decouples the performance process and the learning process. Melody Matrix is not a tool that plays music only. It is a place of collision between technology and creativity to create an unlimited number of ideas in music.

2. Use Cases:

- Guest tries a quick jam (no account)

Actor: Guest User
Goal: Explore the app fast and hear AI-generated music without signing up.

Basic Steps:

- ➢ Guest opens Melody Matrix and chooses “Continue as Guest.”
- ➢ Uploads a MIDI file (or uses a built-in sample).
- ➢ Selects basic controls like tempo (BPM) and mood.
- ➢ Click Play to start a live jam session.
- ➢ Listens to the generated output

Exception:

If a guest user tries to save a model, save a created song, or access history, the system blocks the operation since guest mode is designed to be temporary and does not enable data permanence. Similarly, if a guest attempts to use advanced generation settings such as second- or third-order Markov chains, the system will automatically limit them to first-order mode, ensuring that full functionality is only available to registered users while allowing guests to explore the basic features.

- Registered user creates an account and logs in

Actor: New Registered User
Goal: Get full access (save songs, advanced Markov orders, dashboard/library).

Basic Steps:

- ➢ User clicks Sign Up, enters email/username/password.
- ➢ System validates inputs and creates the account.
- ➢ User logs in and is redirected to their dashboard.
- ➢ System enables “registered-only” actions (save/manage models and compositions).

Exception:

If a user tries to sign up using an existing email address, the system encourages them to log in rather than creating a new account. If the password entered does not meet the required rules, the signup attempt fails, and a clear message explains what needs to be corrected.

- Generate a Multi-Instrument Jam in Real Time

Actor: Guest User or Registered User
Goal: Generate a fresh performance every time and control how it feels.

Basic Steps:

- ➢ User selects a trained model (if logged in) or a demo model (if using guest mode).
- ➢ The user sets generation controls such as BPM, mood, and Markov order. & clicks Play to start generation.

- ➢ The system produces a synchronized performance containing melody, bass, drums, and chords, all aligned to a shared tempo clock so they play together smoothly.
- ➢ While playback is running, the user can adjust BPM or mood to instantly reshape the music output in real time.

Exception:

If playback slows on lower-performance devices, the system reduces generation complexity (for example, fewer notes or simpler patterns) so the audio continues smoothly.

- ## Train a Personal Model from Uploaded MIDI

Actor: Registered User
Goal: Create a personalized music generation model using their own MIDI file and save it for later use.

Basic Steps:

- ➢ User opens the Train Model section & uploads a MIDI file.
- ➢ System reads note data and prepares training input.
- ➢ User selects the smartness level (Markov order 1, 2, or 3).
- ➢ System builds a generation model from the uploaded file & User names the model.
- ➢ System saves it to the user's library.

Exception:

If the uploaded file contains very little musical data, the system allows training to continue but shows a warning that the generated results may be limited or repetitive due to insufficient input.

- ## Download the generated music as a MIDI file (Registered)

Actor: Registered User
Goal: Keep the generated output and reuse it in real music tools

Basic Steps:

- ➢ User generates a music session.
- ➢ Clicks Download MIDI.
- ➢ System packages the generated notes into a MIDI file.
- ➢ User downloads the file and can reuse it later in music software.

Exception:

If music generation is still ongoing, the download option is blocked until the MIDI file is complete. Because guest sessions are temporary, the system may enable download but does not store it to the user library.

3. Users:

- Guest User

  A guest user is a person who uses the platform in a restricted mode of exploration. The main aim of a guest user is to just explore AI-generated music and to skip creating an account. These users may upload MIDI files on a temporary basis, create real-time jam-sessions, change tempo, and hear generated compositions so as to know how the system operates. Non-persistent functionality is, however, the only interaction between them. The guest user does not have the opportunity to save trained models, store generated songs, view analytics history, or provide long-term feedback. Also, they are not in control of accounts and changes in system settings. This restricted entry is used to allow new users to navigate the platform without interfering with data integrity and slowing down database storage.

- Registered User

  A registered user is an authenticated user of the system who uses it as a personalized generative music platform. The main tasks of a registered user is to use the uploaded MIDI data to train AI models, produce original musical pieces, store and manage the ones that are created, analyze statistics of the models they can train, and give feedback on how to improve the quality of their output. Registered users can create, update and delete their own models and compositions, have a personal library of music, a special dashboard and provide ratings or feedback of generated sessions.

  Also the registered users are not allowed to view or edit the models or data of other users, or to change system-level settings, or to do other administrative operations. This is the primary long-term customer of the platform.

- Administrator

  An administrator will manage the work of the system and guarantee reliability, security, and correct use of the platform. The objectives of the administrator is to ensure the stability of the system, user management, tracking the activities of the platform, and ensuring the integrity of the database. To achieve these duties, administrators are given the ability to examine user accounts, disable or delete user accounts as needed, delete damaged or unsuitable content, and use system logs to monitor and perform troubleshooting.

  But , security principles also limit administrative authority. Administrators do not have access to user passwords in plaintext or can do any kind of direct manipulation of the AI model logic unless they have the right development access.

- AI Engine (Internal System Actor)

  The AI Engine is an internal working element that fulfills model training and music generation jobs. Its main goal is to study uploaded MIDI files, calculate transition probabilities, create new compositions based on the acquired patterns, and use user feedback when necessary. In order to do these functions, the AI Engine is allowed to give musical information and work under certain algorithms.

It is limited in its functioning to ensure modularity and safety of the system. The AI Engine does not directly communicative with the users, will not access unrelated user data on its own, and will not override the authentication or administrative control. It is confined to its necessary scope as the smart heart of the platform, where the generation of music is handled in an automated and consistent and disconnected manner of user management issues.

4. Architecture:

Melody Matrix will use client/server architecture. The idea is to currently divide the system into 3 objects: Client, Server, and Database.

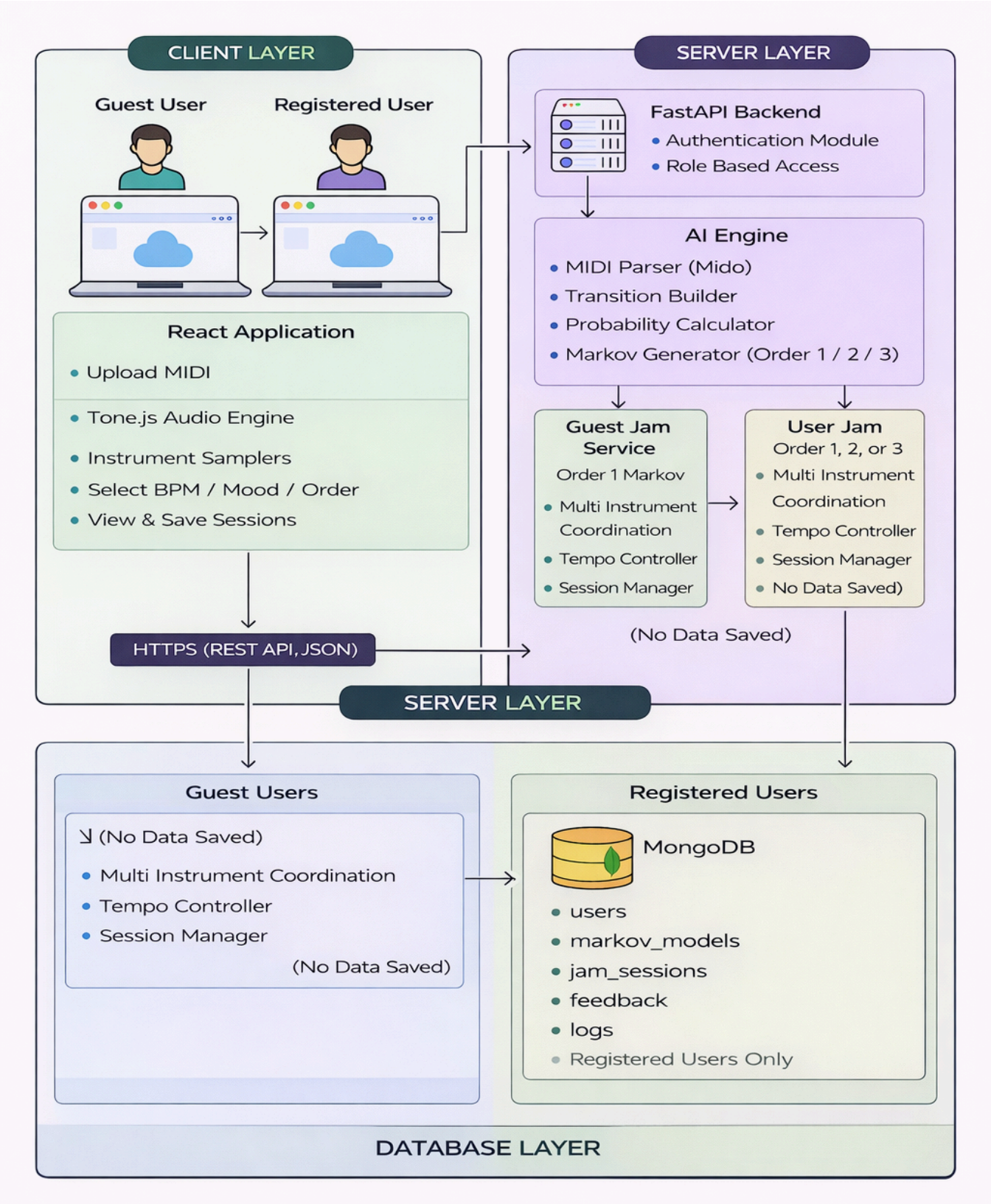
The Client will be written with React, and it will run in the browser and be able to respond to all user interactions. The app will make a distinction between registered and guest users.

Both Registered Users and Guest Users will be able to upload MIDI files and also change tempo and mood, as well as configuration of the Markov order, but some of the features will be restricted to be used only by registered users. Registered users will have more control over the order of Markov chains, the amount of data that can be processed by the server, and other functionalities.

We'll use Tone.js audio engine within the browser, which transforms generated sequences of notes into audio in real time. This architecture allows the client to be involved with interaction and playback.

The Server will deal with authentication, MIDI processing, and Markov chain generation. While the Database using MongoDB will store data of users. It will store previous sessions and a pre-registered functional Markov chain. For example, the user will be able to find the "Blues bass chain" or similar

The chosen architecture is easy to deploy and manage, and it is able to fulfill all the needs of the proposed service.


CLIENT LAYER
Guest User
Registered User
React Application
Upload MIDI
Tone.js Audio Engine
Instrument Samplers
Select BPM / Mood / Order
View & Save Sessions
HTTPS (REST API, JSON)
SERVER LAYER
FastAPI Backend
Authentication Module
Role Based Access
AI Engine
MIDI Parser (Mido)
Transition Builder
Probability Calculator
Markov Generator (Order 1 / 2 / 3)
Guest Jam Service
Order 1 Markov
Multi Instrument Coordination
Tempo Controller
Session Manager
User Jam
Order 1, 2, or 3
Multi Instrument Coordination
Tempo Controller
Session Manager
No Data Saved)
(No Data Saved)
SERVER LAYER
Guest Users
(No Data Saved)
Multi Instrument Coordination
Tempo Controller
Session Manager
(No Data Saved)
Registered Users
MongoDB
users
markov_models
jam_sessions
feedback
logs
Registered Users Only
DATABASE LAYER

5. Technologies:

The technical core of the Markov Chain Music Creator relies on a unified pipeline where Python-based analysis transforms MIDI data into structured probability maps for real-time web playback. By utilizing Python and its specialized libraries like Mido, the application can deconstruct complex MIDI files into discrete musical events such as pitch, velocity, and duration. These events are processed through a transition frequency algorithm that calculates the statistical likelihood of one note following another. This transformation turns a linear song into a generative "map" where the next note is always a probabilistic choice rather than a fixed sequence.

To manage these transition maps, MongoDB provides a flexible document storage layer that naturally handles the nested, non-linear structure of a Markov matrix. Instead of forcing musical relationships into rigid tables, each Markov Chain is stored as a single BSON document containing the adjacency list of notes and their associated weights. This allows the system to store both user-generated models and the "instrument database" presets, like a C-scale guitar or a foundational bass rhythm, as easily accessible JSON objects that the frontend can fetch in a single query.

The final stage of the architecture involves a high-performance frontend built with React and Tone.js. The browser acts as the synthesizer, receiving the transition matrix from MongoDB and using it to trigger high-fidelity audio samples. By hosting a library of sampled instrument sounds, such as a clean electric guitar or a deep synth bass, the Web Audio API can perform the Markov Chain's output with professional-grade sound quality. This setup ensures that the heavy computation and data organization happen on the backend while the creative, interactive performance happens instantly on the user's device.

6. Team Skill set:

Mattia: Python, Docker, Git, Markov chains
Mansi: Python, MongoDB, Git
Nanditha: Python, SQL, Git
Sashank: Python, HTML, CSS, Node JS
Vignesh: HTML, CSS, Python

7. Challenges and Risks:

In Melody Matrix, one problem is that repeated music patterns may occur, particularly with training on small datasets. A Markov Chain can vary less because the probabilities are learned, so fewer input steps are required. This can be enhanced in the future through updating with higher order models and more variation controls.

The other risk is that low-end devices will have limitations in their performance because the browser will be operating real-time audio with Tone.js. This can even create minor latency. This may be overcome through optimization of audio scheduling and simplification of generation dynamically.

Another problem is the quality of uploaded MIDI files. Less rich music can be created with simple or short MIDI inputs. This can be resolved with recommendations on training, in-built data sets, and style-blending in future releases.

Timing mismatch between melody, bass, and drum agents is also a minor risk. They may vary slightly in terms of timing, although they have the same tempo. The tightening of the synchronization will be guaranteed by improvement of the timing control in the future.

Lastly, the increase in the number of users who save models can increase the database size and potentially influence performance. This is to be addressed in future through database indexing, optimization and scalable cloud storage system.

8. Timeline:

The following timeline represents the project timeline we have structured for our development phases. We are committed to following this plan to ensure organized, efficient, and timely execution of all tasks.

Phase /	Sprint 1	Sprint 2	Sprint 3	Sprint 4
Requirements & Music Theory Modeling				
Markov Model Development				
Client App (Guest/Registered) Implementation				
API & Session Engine				
Evaluation & User Testing				
Deployment & Demo				